از صفر تا صد نمایش متن روی LCD دو در شانزده

شرح کامل منطق برنامه‌ی STM32 با TouchGFX و FreeRTOS، برای کسی که از هیچ شروع می‌کند

۰. این سند چه چیزی به شما یاد می‌دهد؟

فرض این سند این است که شما هیچ دانش قبلی ندارید. هدف این است که پس از خواندنش نه‌تنها بفهمید این برنامه چه می‌کند، بلکه بتوانید برنامه‌های متفاوت دیگری هم بنویسید، مثلاً نمایش عدد یک سنسور، یک شمارنده یا پیام‌های دلخواه با دکمه‌های بیشتر.

برنامه چه کاری انجام می‌دهد؟

روی یک صفحه‌ی لمسی بزرگ (۱۰۲۴ در ۶۰۰) سه دکمه وجود دارد: **Test**، **STM32** و **Clear**. وقتی انگشتتان یکی از دکمه‌ها را لمس می‌کند، یک متن کوتاه روی یک نمایشگر کاراکتری کوچک **دو سطری و شانزده ستونی** (LCD 2x16) نوشته می‌شود. دکمه‌ی Test کلمه‌ی «Test»، دکمه‌ی STM32 کلمه‌ی «STM32» و دکمه‌ی Clear متن را خالی می‌کند. این‌ها دو نمایشگر کاملاً جدا هستند: یکی تصویری و لمسی، دیگری فقط حروف.

> **ایده‌ی اصلی در یک جمله**
>
> بخشی که لمس را می‌فهمد فقط یک «پیام» (عدد ۱، ۲ یا ۳) می‌فرستد و بخش جداگانه‌ای که مسئول LCD است، این پیام را می‌گیرد، متن را تعیین می‌کند و با ترتیب دقیق و مطمئن روی نمایشگر می‌نویسد.

نقشه‌ی راه سند

فصل	موضوع	چرا لازم است؟
۱	مفاهیم پایه‌ی زبان C که در این کد دیده می‌شود	بدون آن‌ها کد خوانده نمی‌شود
۲	سخت‌افزار: LCD، تراشه‌ی ۷۴HC595 و SPI	بفهمید داده چگونه به نمایشگر می‌رسد
۳	دستورهای نمایشگر و ترتیب راه‌اندازی آن	منطق LCD_Init و SetCursor
۴	سیستم‌عامل FreeRTOS: تسک و صف پیام	منطق Start_lcd_Task
۵	مسیر کامل از لمس تا نوشتن روی LCD	دید کلی از کل برنامه
۶	شرح خط‌به‌خط کدهای اصلی	هسته‌ی این سند
۷	چرا هر بار راه‌اندازی دوباره؟ و باگ‌های نسخه‌ی قدیمی	تصمیم‌های طراحی
۸	الگوی ساخت برنامه‌های دیگر (آماده برای استفاده)	تبدیل دانش به مهارت
۹	عیب‌یابی و واژه‌نامه	مرجع سریع

کدهایی که در این سند کامل توضیح داده می‌شوند

```
static const char *line1 = "";
static const char *line2 = "";

static void LCD_PrintLine(uint8_t row, const char *s) { ... }
static void LCD_Refresh(void) { ... }
void Start_lcd_Task(void *argument) { ... }
```

این چهار قطعه با هم «منطق نمایش» را می‌سازند. در فصل ۶ همه‌ی آن‌ها را خط‌به‌خط می‌خوانیم.

۱. مفاهیم پایه‌ی C که لازم دارید

میکروکنترلر (مثل STM32) یک کامپیوتر کوچک است که فقط یک برنامه را اجرا می‌کند. ما برنامه را به زبان C می‌نویسیم، کامپایلر آن را به کد ماشین تبدیل می‌کند و آن را روی تراشه می‌ریزیم. هشت مفهومی که برای خواندن این پروژه کافی است:

۱-۱ متغیر

متغیر یک «جعبه‌ی نام‌دار» در حافظه است. `uint8_t row` یعنی جعبه‌ای به نام row که عددی بین ۰ تا ۲۵۵ در آن جا می‌شود. `uint32_t command` عددی بسیار بزرگ‌تر (تا حدود چهار میلیارد) نگه می‌دارد. حرف u یعنی «بدون علامت» و عدد بعدی تعداد بیت‌هاست.

۱-۲ تابع

تابع یک «دستورالعمل نام‌دار» است. `LCD_Refresh()` کارهای متعددی را پشت یک نام جمع می‌کند و هر جا که صدا بزنید همه را به‌ترتیب انجام می‌دهد. کلمه‌ی `void` یعنی چیزی برنمی‌گرداند و داخل پرانتز، ورودی‌هاست.

۱-۳ حلقه‌ی for و if

`for (i = 0; i < 16; i++)` یعنی «شانزده بار تکرار کن». `if (شرط) { ... }` یعنی «فقط اگر شرط درست بود انجام بده». `switch (command)` مثل چند if پشت سرهم است و با هر مقدار، یک شاخه‌ی case را اجرا می‌کند و با `break` از آن بیرون می‌آید.

۱-۴ متن (رشته) در C

در C متن یعنی «ردیفی از حروف که با کاراکتر ویژه‌ی صفر (`'\0'`) تمام می‌شود». پس `"Test"` در حافظه می‌شود: T، e، s، t، و صفر پایانی. برنامه با دیدن همین صفر می‌فهمد متن تمام شده است.

۱-۵ اشاره‌گر (pointer) به متن

`const char *line1` یعنی «line1 آدرس اولین حرف یک متن را نگه می‌دارد». خود متن را کپی نمی‌کند، فقط می‌گوید کجا قرار دارد. وقتی می‌نویسیم `line1 = "Test";` فقط آدرس عوض می‌شود. چون متن‌های داخل گیومه در حافظه‌ی دائمی برنامه (فلش) هستند، تا آخر عمر برنامه سر جایشان می‌مانند و این کار ایمن است. علامت `*s` یعنی «حرفی که s به آن اشاره می‌کند» و `s++` یعنی «برو به حرف بعدی».

۱-۶ کلیدواژه‌ی const و static

`const` یعنی «محتوا قابل تغییر نیست» (خود متن ثابت است، ولی می‌توانیم آدرس را به متن دیگری ببریم). `static` بیرون از تابع یعنی «این متغیر یا تابع فقط داخل همین فایل دیده شود»، که از قاطی شدن نام‌ها جلوگیری می‌کند.

۱-۷ عملگر شرطی؟:

`*s ? *s++ : ' '` یک if فشرده است: «اگر حرف فعلی صفر نبود همان را بده و برو حرف بعد؛ وگرنه فاصله بده». این خط دقیقاً همان کاری است که LCD_PrintLine برای پر کردن با فاصله انجام می‌دهد.

۱-۸ اعداد ۱۶تایی (hex)

دستورهای LCD به شکل `0x28` و `0x0C` نوشته می‌شوند. پیشوند `0x` یعنی عدد در مبنای شانزده است. نیازی به محاسبه‌ی ذهنی نیست، فقط بدانید که هر رقم hex دقیقاً چهار بیت (یک «نیبل») را نشان می‌دهد. این موضوع در فصل ۲ مهم می‌شود.

> **تمرین کوتاه**
> اگر `s` به متن "Hi" اشاره کند و حلقه‌ی شانزده‌تایی اجرا شود، چه می‌شود؟ جواب: ابتدا H و i نوشته می‌شوند. بعد `*s` صفر می‌شود و ۱۴ بار فاصله نوشته می‌شود. در نتیجه ۱۶ خانه‌ی کامل پر شده و هیچ حرف قدیمی باقی نمی‌ماند.

۲. سخت‌افزار: داده چگونه به LCD می‌رسد؟

۲-۱ نمایشگر کاراکتری دو در شانزده

این نمایشگر شانزده خانه در هر سطر و دو سطر دارد (جمعاً ۳۲ خانه). داخلش یک تراشه‌ی استاندارد به نام **HD44780** است که نقشه‌ی شکل حروف را می‌داند. ما فقط «کد حرف» را می‌فرستیم (مثلاً `0x54` برای T) و تراشه خودش شکلش را رسم می‌کند. این تراشه دو نوع ورودی می‌پذیرد:

- **دستور (Command)**: مثل «صفحه را پاک کن» یا «مکان‌نما را ببر به ابتدای سطر دوم».
- **داده (Data)**: کد یک حرف که باید در مکان فعلی نوشته شود.

پایه‌ی **RS** فرق این دو را می‌گوید (RS=0 یعنی دستور، RS=1 یعنی داده). پایه‌ی **EN** (Enable) مثل یک «دکمه‌ی تأیید» است: هر وقت EN از ۱ به ۰ برود، تراشه آنچه روی پایه‌های داده می‌بیند را قبول می‌کند.

۲-۲ حالت چهاربیتی

پایه‌های داده‌ی تراشه هشت‌تا هستند (D0 تا D7)، ولی ما فقط چهارتای بالا (D4 تا D7) را سیم‌کشی می‌کنیم تا سیم کمتری لازم باشد. پیامد این کار: برای فرستادن یک بایت (هشت بیت) باید آن را در **دو نوبت** بفرستیم، اول چهار بیت بالا و بعد چهار بیت پایین. به هر چهار بیت یک «نیبل» (nibble) می‌گویند. در کد، `value >> 4` نیبل بالا و `value & 0x0F` نیبل پایین را جدا می‌کند.

۲-۳ چرا ۷۴HC595؟

میکروکنترلر پایه‌های آزاد کمی دارد. تراشه‌ی ۷۴HC595 یک «مبدل سریال به موازی» است: ما یک بایت را با دو سیم (کلاک و داده) پشت سر هم می‌فرستیم و این تراشه آن را روی هشت خروجی موازی می‌گذارد. برای تأیید نهایی هم یک سیم سوم لازم است که به آن «latch» می‌گویند. در این پروژه سه سیم از میکروکنترلر استفاده می‌شود:

پایه‌ی STM32	نقش
PA5 (SPI1_SCK)	کلاک: هر پالس یک بیت را به داخل ۵۹۵ هل می‌دهد
PA7 (SPI1_MOSI)	داده: بیت‌ها از اینجا ارسال می‌شوند
PA4 (SPI1_CS)	در کد ما نقش latch دارد: وقتی از صفر به یک برگردد، مقدار فرستاده‌شده روی خروجی‌های ۵۹۵ ظاهر می‌شود

پروتکل **SPI** فقط یک قرارداد برای فرستادن بیت‌هاست. توجه کنید که کد با تابع `HAL_SPI_Transmit` یک بایت را می‌فرستد و CS را قبل و بعد از آن دستی صفر و یک می‌کند. این همان تابع `LCD_Shift595` است.

۲-۴ هر بیت از بایت ۵۹۵ به کجا وصل است؟

بایتی که به ۵۹۵ می‌فرستیم هشت بیت دارد و هر بیت یک خروجی را کنترل می‌کند. طبق ثابت‌های کد (`LCD_RS_BIT` برابر بیت ۴ و `LCD_EN_BIT` برابر بیت ۵) و فرض سیم‌کشی استاندارد، نقشه چنین است:

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	EN	RS	D7	D6	D5	D4

چهار بیت پایین، نیبل داده را حمل می‌کنند. بنابراین برای فرستادن یک نیبل مثلاً ۰x۵ به‌صورت «داده» باید بایت `0x05 | RS` را ساخت، و برای تأیید آن، EN را یک و بعد صفر کرد. دقیقاً همین کار را `LCD_Write4` انجام می‌دهد.

> **نکته‌ی سخت‌افزاری مهم**
> اگر ۷۴HC595 با تغذیه‌ی ۵ ولت کار کند، خروجی ۳.۳ ولتی STM32 به حداقل ولتاژ «یک» شناخته‌شده‌ی آن (حدود ۳.۱۵ ولت) خیلی نزدیک است و با گرم شدن یا نویز، کار ناپایدار می‌شود. از ۷۴HCT595 استفاده کنید یا آن را با ۳.۳ ولت تغذیه کنید. یک خازن ۱۰۰ نانوفاراد نزدیک پایه‌ی تغذیه‌ی ۵۹۵ و LCD هم لازم است.

۳. دستورهای نمایشگر و ترتیب راه‌اندازی

۳-۱ از یک نیبل تا یک حرف

یک حرف روی LCD چنین فرستاده می‌شود (تابع‌های `LCD_Write4` و `LCD_Send`):

1. نیبل بالا را به ۵۹۵ می‌دهیم همراه EN=1 و RS مناسب.
2. کمی صبر می‌کنیم (یک میکروثانیه).
3. همان نیبل را با EN=0 می‌فرستیم. این لبه‌ی پایین‌رونده، نیبل را وارد LCD می‌کند.
4. همین سه مرحله را برای نیبل پایین تکرار می‌کنیم.
5. حدود ۵۰ میکروثانیه صبر می‌کنیم تا LCD کارش را تمام کند.

تراشه‌ی HD44780 کند است. اگر قبل از تمام شدن یک کار دستور بعدی برسد، آن را از دست می‌دهد. به همین دلیل بعد از هر ارسال تأخیر داریم و بعد از دستور «پاک کردن صفحه» بیش از همه (حدود ۲ میلی‌ثانیه).

۳-۲ دستورهای مهمی که در کد می‌بینید

مقدار	معنی
0x28	حالت چهاربیتی، دو سطره، قلم ۵×۸ (به این دستور «Function Set» می‌گویند)
0x0C	نمایشگر روشن، مکان‌نمای چشمک‌زن خاموش
0x06	بعد از نوشتن هر حرف، مکان‌نما یک خانه به راست برود
0x01	پاک کردن کل صفحه (کند: حدود ۱.۵ میلی‌ثانیه)
0x80 | آدرس	گذاشتن مکان‌نما روی خانه‌ای مشخص (Set DDRAM Address)

۳-۳ آدرس خانه‌ها و تابع LCD_SetCursor

حافظه‌ی نمایش LCD یک آدرس‌دهی مخصوص دارد: سطر اول از آدرس `0x00` شروع می‌شود و سطر دوم از `0x40` . پس خانه‌ی ستون ۵ از سطر دوم آدرس `0x40 + 5` است. برای گذاشتن مکان‌نما بیت هفتم باید یک باشد، یعنی دستور = `0x80 | (0x40 + 5)` . تابع `LCD_SetCursor(row, col)` دقیقاً همین را حساب می‌کند.

```
address = (row == 0) ? 0x00 : 0x40;
LCD_WriteCommand(0x80 | (address + col));
```

۳-۴ راه‌اندازی اولیه (LCD_Init)

هنگام روشن شدن، تراشه در حالت هشت‌بیتی است و نمی‌دانیم کجای یک بایت هستیم. برای اینکه از هر وضعیتی، دقیقاً به حالت چهاربیتی برسیم، سازنده یک دعای خاص تعیین کرده است:

1. کمی صبر (۵۰ میلی‌ثانیه) تا ولتاژ پایدار شود.
2. نیبل `0x3` را سه بار می‌فرستیم (با تأخیرهای ۵، ۱ و ۱ میلی‌ثانیه). این «بازنشانی با دستور» است و هر حالتی را به هشت‌بیتی معلوم برمی‌گرداند.
3. نیبل `0x2` را می‌فرستیم تا رسماً به حالت چهاربیتی برود.
4. سپس دستورهای `0x28` ، `0x0C` ، `0x06` و در پایان `0x01` (پاک کردن).

> **چرا این ترتیب بسیار مهم است؟**
> چون LCD چهاربیتی هیچ‌گاه نمی‌گوید «الان کدام نیبل را از من خواستی؟». اگر فقط یک نیبل در مسیر گم یا اضافه شود، از آن به بعد نیبل‌های بالا و پایین جابه‌جا می‌شوند و فقط آشغال یا صفحه‌ی خالی می‌بینید. ترتیب بالا تنها راه مطمئن برای برگرداندن همه‌چیز به حالت درست است و در فصل ۷ از همین ویژگی برای «خوددرمانی» استفاده می‌کنیم.

۳-۵ تأخیر دقیق به‌جای حلقه‌ی خالی

تابع `LCD_DelayUs` باید دقیقاً چند میکروثانیه صبر کند. نسخه‌ی قدیمی با یک حلقه‌ی خالی این کار را می‌کرد که دو ایراد داشت: زمانش بسیار بیشتر از انتظار بود و با بهینه‌سازی کامپایلر کلاً حذف می‌شد. نسخه‌ی جدید از شمارنده‌ی سخت‌افزاری چرخه‌های پردازنده (DWT) استفاده می‌کند. چون پردازنده‌ی ما ۲۵۰ مگاهرتز است، هر میکروثانیه ۲۵۰ چرخه است و شمارنده فقط تا آن عدد بالا می‌رود.

۴. سیستم‌عامل FreeRTOS: تسک و صف پیام

۴-۱ تسک (Task) چیست؟

یک تسک، یک «برنامه‌ی کوچک مستقل» است که برای خودش یک حلقه‌ی بی‌پایان دارد. FreeRTOS با سرعت بسیار بالا بین چند تسک جابه‌جا می‌شود، به‌طوری که به نظر می‌رسد همه‌ی آن‌ها هم‌زمان کار می‌کنند. هر تسک یک **اولویت** دارد و اگر تسک مهم‌تری آماده‌ی اجرا شود، تسک کم‌اهمیت‌تر را موقتاً کنار می‌زند. این پروژه سه تسک دارد:

تسک	اولویت	کار
`GUITask`	بالا	رسم صفحه‌ی لمسی، خواندن لمس، اجرای کد دکمه‌ها (TouchGFX)
`lcd_Task`	پایین	تنها تسکی که با LCD دو در شانزده کار دارد (موضوع این سند)
`defaultTask`	معمولی	فعلاً کاری ندارد و فقط جا را پر می‌کند

چرا lcd_Task اولویت پایین دارد؟ چون رسم تصویر مهم است و نباید به خاطر نوشتن آرام روی یک LCD کاراکتری ببندد.

۴-۲ چرا مستقیم از دکمه روی LCD ننویسیم؟

ممکن بود در تابع دکمه مستقیم `LCD_WriteString` را صدا بزنیم. اما نوشتن روی LCD با تأخیرهای آن ده‌ها میلی‌ثانیه طول می‌کشد و GUI را قفل می‌کند. همچنین اگر دو بخش هم‌زمان LCD را دست بزنند، نیبل‌ها قاطی می‌شوند. قاعده‌ی طلایی این است:

> **قاعده‌ی طلایی**
> فقط یک تسک اجازه‌ی دست زدن به LCD را دارد. بقیه فقط برایش «پیام» می‌فرستند.

۴-۳ صف پیام (Message Queue)

صف مثل یک «صندوق نامه» است. یک طرف نامه می‌اندازد (`osMessageQueuePut`) و طرف دیگر نامه را برمی‌دارد (`osMessageQueueGet`). در این پروژه صف `lcdQueue` ۳۲ جا دارد و هر جا یک عدد چهاربایتی (`uint32_t`) نگه می‌دارد. قرارداد پیام‌ها این است:

عدد پیام	معنی	دکمه‌ی فرستنده
1	سطر اول را «Test» کن	`btntest`
2	سطر اول را «STM32» کن	`btnstm32`
3	سطر اول را خالی کن	`btnclear`

۴-۴ انتظار با مهلت: قلب Start_lcd_Task

خط مهم این است:

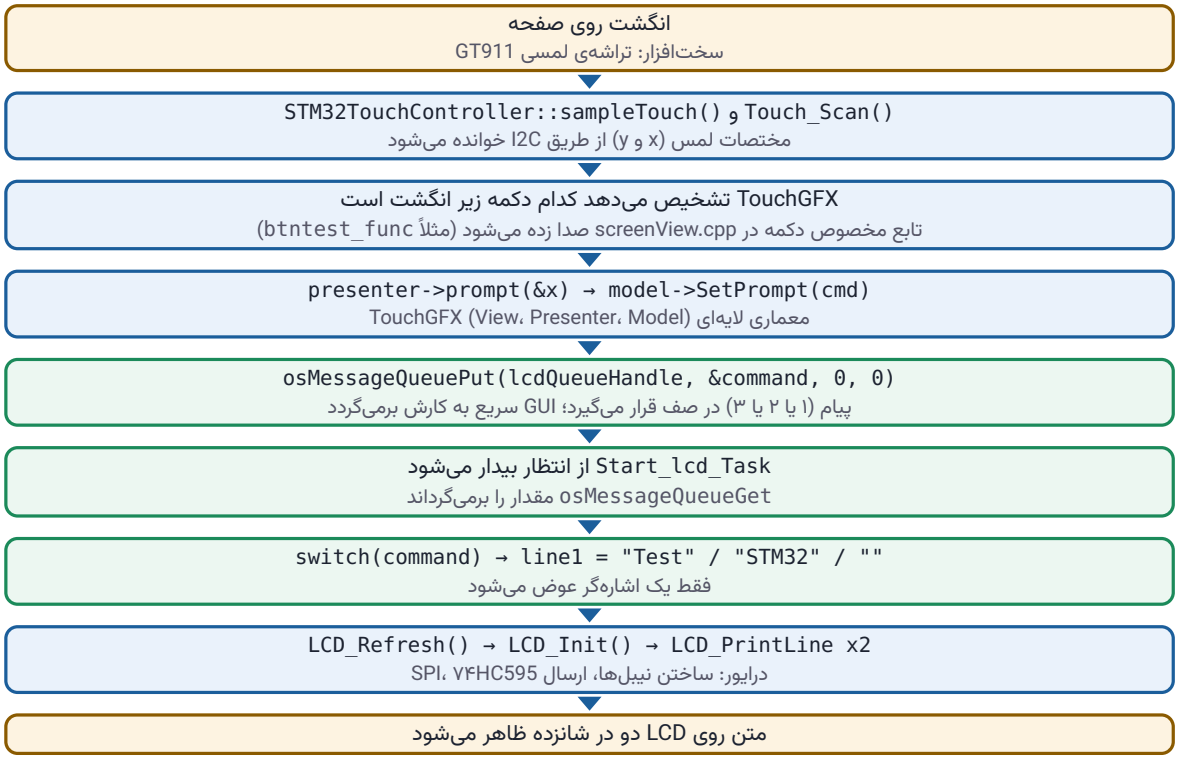
```
osMessageQueueGet(lcdQueueHandle, &command, NULL, 2000)
```

یعنی «در صندوق نگاه کن؛ اگر پیامی بود، همین الان آن را در `command` بریز و `osOK` برگردان. اگر خالی بود، تا ۲۰۰۰ میلی‌ثانیه (۲ ثانیه) منتظر بمان. اگر باز هم خالی بود، با کد خطای timeout برگرد». نکته‌ی زیبا این است که در مدت انتظار، تسک خوابیده است و هیچ پردازشی مصرف نمی‌کند و CPU به تسک‌های دیگر می‌رسد. پس این یک «بیدارباش دوگانه» است: یا پیام آمده یا ۲ ثانیه گذشته.

> **اشتباه رایجی که نسخه‌ی قدیمی داشت**
>
> نسخه‌ی قدیمی از `osMessageQueueGet(..., 0, 0)` (بدون انتظار) استفاده می‌کرد و پس از آن حتی اگر صف خالی بود، همان `command` قبلی را دوباره اجرا می‌کرد. چون در حالت خالی بودن، تابع هیچ مقداری در command نمی‌نویسد، مقدار قدیمی می‌ماند و برنامه برای همیشه تکرار می‌کرد. اکنون فقط وقتی `== osOK` شود، پیام جدید پردازش می‌شود.

۵. مسیر کامل از لمس تا نوشتن روی LCD

هر بار که دکمه‌ای را لمس می‌کنید، این زنجیره‌ی کامل از رویدادها اتفاق می‌افتد. این نمودار را بارها نگاه کنید. دانستنش مهم‌ترین چیز برای نوشتن برنامه‌های دیگر است.



۵-۱ لایه‌ها و مسئولیت هر کدام

لایه	مسئولیت	فایل
رابط کاربری	رسم دکمه‌ها و فهمیدن لمس. هیچ چیز درباره‌ی LCD نمی‌داند.	`screenView.cpp`
میانجی	گذاشتن پیام در صف.	`Model.cpp`
منطق نمایش	تصمیم‌گیری درباره‌ی «چه چیزی باید نوشته شود».	`main.c (Start_lcd_Task)`
درایور LCD	«چطور» نوشتن: نیبل، RS، EN، تأخیر.	`lcd.c`
سخت‌افزار	SPI و ۷۴HC595 و LCD.	برد و HAL

چرا این‌همه لایه؟
چون هر لایه فقط به لایه‌ی کناری خودش وابسته است. می‌توانید LCD را با یک نمایشگر دیگر عوض کنید و فقط lcd.c را تغییر دهید، بدون اینکه دکمه‌ها را لمس کنید. یا بخواهید به‌جای لمس از یک سنسور پیام بفرستید و هیچ چیز از LCD تغییر نکند. این همان راز نوشتن برنامه‌های بزرگ است.

۵-۲ یک جزئیات جالب: چرا x سراسری است؟

در `screenView.cpp` متغیر سراسری `int x` وجود دارد. دکمه‌ها مقدار آن را عوض می‌کنند و آدرسش را به presenter می‌دهند. این روش کار می‌کند، ولی ساده‌تر و بی‌خطرتر این است که خود عدد را مستقیم بفرستید (`prompt(1)`). در هر دو حالت `Model::SetPrompt` مقدار را در یک متغیر محلی `command` کپی می‌کند و `osMessageQueuePut` نیز یک نسخه‌ی مستقل از آن را در صف می‌گذارد.

۶. شرح خط‌به‌خط کدهای اصلی

۶-۱ دو متغیر سراسری برای متن‌ها

```
static const char *line1 = "";
static const char *line2 = "";
```

- `line1` و `line2` دو «برگه‌ی یادداشت» هستند. در هر کدام نوشته شده «متنی که باید روی سطر اول و دوم باشد کجاست».
- مقدار اولیه‌ی `""` متن خالی است (فقط یک صفر پایانی). یعنی در ابتدا هیچ‌چیز نمایش داده نمی‌شود.
- `static` : فقط داخل همین فایل دیده می‌شوند و بیرون آن به آن‌ها دست نمی‌زنند.
- هر وقت بخواهیم محتوای LCD عوض شود، فقط همین دو یادداشت را عوض می‌کنیم و بقیه خودش پیش می‌رود. این «جداسازی داده از نمایش» است.

۶-۲ تابع LCD_PrintLine: نوشتن کامل یک سطر

```
static void LCD_PrintLine(uint8_t row, const char *s)
{
    LCD_SetCursor(row, 0);                      // (1)
    for (uint8_t i = 0; i < 16; i++)          // (2)
        LCD_WriteChar(*s ? *s++ : ' ');        // (3)
}
```

خط ۱: مکان‌نما را به ابتدای سطر موردنظر می‌برد (row صفر یعنی سطر بالا، یک یعنی پایین؛ ستون صفر یعنی اول سطر).

خط ۲: دقیقاً شانزده بار تکرار می‌کند، یک بار برای هر خانه از سطر.

خط ۳: اگر حرف فعلی صفر نیست (یعنی متن هنوز تمام نشده)، همان حرف را می‌نویسد و به حرف بعدی می‌رود. وگرنه یک فاصله می‌نویسد. به همین دلیل همیشه ۱۶ خانه‌ی کامل پر می‌شود.

چرا فاصله می‌نویسیم؟
فرض کنید قبلاً «STM32» نوشته شده و حالا «Test» را می‌نویسیم. بدون پر کردن، نتیجه «Test2» می‌شد (حرف ۲ از متن قبلی می‌ماند). با فاصله‌ها، همه‌ی خانه‌های باقی‌مانده پاک می‌شوند و هیچ بقایایی نمی‌ماند. همین ترفند مشکل «اطلاعات اشتباه» را حل می‌کند. توجه کنید که متن‌های بلندتر از ۱۶ حرف، ۱۶ حرف اول‌شان نمایش داده می‌شود و بقیه نادیده گرفته می‌شود.

۶-۳ تابع LCD_Refresh: بازسازی کامل نمایشگر

```
static void LCD_Refresh(void)
{
    LCD_Init();                 // (1) resync with the display
    LCD_PrintLine(0, line1);    // (2) top row
    LCD_PrintLine(1, line2);    // (3) bottom row
}
```

خط ۱: راه‌اندازی کامل (فصل ۳-۴) را از نو انجام می‌دهد. این کار صفحه را هم پاک می‌کند و از هر حالت خرابی، آن را به وضعیت درست می‌رساند.

خط ۲ و ۳: محتوای کامل دو سطر را از «یادداشت‌ها» می‌خواند و می‌نویسد.

پس `LCD_Refresh` دو چیز را تضمین می‌کند: هر آنچه روی LCD است دقیقاً برابر `line1` و `line2` است، و خودِ LCD سالم و هم‌زمان است.

> **هزینه‌ی این روش**
> هر Refresh حدود ۸۰ میلی‌ثانیه طول می‌کشد و در انتهای راه‌اندازی، صفحه برای چند میلی‌ثانیه پاک و دوباره نوشته می‌شود. ممکن است هر ۲ ثانیه یک «پلک‌زدن» بسیار ریز ببینید. اگر آزارتان داد، عدد ۲۰۰۰ را در Start_lcd_Task به ۵۰۰۰ یا ۱۰۰۰۰ تغییر دهید.

۶-۴ تابع Start_lcd_Task: مغز برنامه

```
void Start_lcd_Task(void *argument)
{
    uint32_t command;                                   // (1)
    line1 = "Working...";                               // (2)
    LCD_Refresh();                                      // (3)

    for (;;)                                            // (4)
    {
        if (osMessageQueueGet(lcdQueueHandle, &command,
                              NULL, 2000) == osOK)      // (5)
        {
            switch (command)                            // (6)
            {
            case 1: line1 = "Test";  break;
            case 2: line1 = "STM32"; break;
            case 3: line1 = "";      break;
            default: break;                             // (7)
            }
        }
        LCD_Refresh();                                  // (8)
    }
}
```

خط	شرح
(1)	یک جعبه‌ی محلی برای دریافت پیام. هنوز مقدار ندارد، ولی طبق منطق تنها وقتی خوانده می‌شود که پیام واقعی رسیده باشد.
(2)	متن شروع را روی «Working...» می‌گذاریم تا پس از روشن شدن مطمئن شویم LCD کار می‌کند.
(3)	اولین نوشتن روی LCD: راه‌اندازی و نمایش «Working...». این اولین باری است که LCD روشن می‌شود.
(4)	`for(;;)` حلقه‌ی بی‌پایان است. تسک‌ها هرگز نباید تمام شوند.
(5)	انتظار تا ۲ ثانیه برای پیام. اگر پیام رسید و `osOK` بود وارد بلوک می‌شویم. اگر نرسید، بلوک رد می‌شود و فقط خط ۸ اجرا می‌شود.
(6)	بر اساس عدد پیام، `line1` را عوض می‌کنیم. فقط اشاره‌گر عوض می‌شود، هنوز چیزی روی LCD نیامده است.
(7)	پیام‌های ناشناخته کاری نمی‌کنند. ایمنی در برابر خطا.
(8)	هم پس از پیام و هم پس از ۲ ثانیه بی‌پیامی اجرا می‌شود. اولی «نمایش تغییر» و دومی «خوددرمانی دوره‌ای» است.

۶-۵ یک لحظه‌ی کامل را قدم‌به‌قدم دنبال کنیم

فرض کنید دستگاه تازه روشن شده و شما دکمه‌ی Test را می‌زنید:

1. روشن شدن: متن `line1` برابر «Working...» می‌شود و `LCD_Refresh` اجرا می‌شود. LCD عبارت «Working...» را نشان می‌دهد.

2. حلقه شروع می‌شود و در خط ۵ منتظر می‌ماند (حداکثر ۲ ثانیه).
3. دو ثانیه بدون پیام می‌گذرد، timeout رخ می‌دهد و Refresh دوباره همان «Working...» را می‌نویسد (بازسازی دوره‌ای).
4. شما Test را می‌زنید. GUI عدد ۱ را در صف می‌گذارد. lcd_Task فوراً بیدار می‌شود.
5. در `switch` مقدار ۱ می‌آید و `line1="Test"` می‌شود.
6. Refresh: راه‌اندازی، سطر اول = «۱۲» + «Test» فاصله، سطر دوم = ۱۶ فاصله. روی LCD فقط «Test» دیده می‌شود.
7. برنامه به خط ۵ برمی‌گردد و همین‌طور ادامه می‌دهد.

۷. چرا هر بار راه‌اندازی دوباره؟ و باگ‌های نسخه‌ی قدیمی

۷-۱ خوددرمانی: منطق پشت LCD_Refresh

در برنامه‌ی اولیه، LCD فقط یک بار راه‌اندازی می‌شد. دستگاه ممکن بود ده دقیقه خوب کار کند و بعد (با فشردن دکمه) متن غلط یا هیچ نمایش داده شود. دلیل احتمالی: در آن ده دقیقه نویز الکتریکی یا افت لحظه‌ای تغذیه یا یک خطای کوچک، حالت LCD یا هم‌زمانی نیبل‌ها را به‌هم زده بود و چون LCD چهاربیتی هیچ بازخوردی نمی‌دهد، میکروکنترلر نمی‌فهمید.

راه‌حل مهندسی: «اگر نمی‌توانی خطا را تشخیص بدهی، حالت را مرتب بازسازی کن.» با `LCD_Init` در هر Refresh، هر مشکلی حداکثر تا ۲ ثانیه بعد خودش درست می‌شود. این اصل را «طراحی مقاوم» یا «سلف‌هیلینگ» می‌گویند و در محصولات صنعتی همه‌جا استفاده می‌شود.

> **صادقانه**
> من علت دقیق مشکل «بعد از ۱۰ دقیقه» را بدون آزمایش روی خود برد نمی‌توانم قطعی بگویم. این روش علت را درمان نمی‌کند، بلکه نتیجه‌اش را بی‌اثر می‌کند. اگر پس از اعمال آن هنوز LCD گاهی خراب می‌شود، سخت‌افزار را بررسی کنید (تغذیه‌ی ۵۹۵، خازن‌ها، سیم‌های کوتاه).

۷-۲ باگ‌های نسخه‌ی قدیمی که اصلاح شدند

مشکل	توضیح	راه‌حل
`command` بدون مقداردهی و تکراری	در حالت خالی بودن صف، مقدار قدیمی دوباره اجرا می‌شد و اولین بار آشغال بود.	پردازش فقط وقتی `== osOK`
متن‌ها روی هم می‌افتادند	«Test» روی «Working...» می‌شد «Testing...» و روی «STM32» می‌شد «Test2».	پر کردن همه‌ی ۱۶ خانه با فاصله
تأخیر ۵۰ میکروثانیه‌ای در واقع حدود ۱۳ میلی‌ثانیه	ضرب ۱۰۰۰ و ۱۰ برابر کردن حلقه؛ سرریز `uint16_t` برای بیش از ۶۵ میکروثانیه.	تأخیر دقیق با شمارنده‌ی DWT
خروجی SPI چک نمی‌شد	یک نیبل گم‌شده، هم‌زمانی را برای همیشه به‌هم می‌ریخت.	راه‌اندازی دوباره در Refresh
شمارش رفرش SDRAM (917)	برای ساعت ۱۲۰ مگاهرتز بود، اما ساعت واقعی ۱۰۰ مگاهرتز است و رفرش خیلی کند می‌شد.	مقدار حدود ۷۴۰

۷-۳ جایی که کد را بگذارید

کدهای Start_lcd_Task و توابع کمکی باید داخل بخش‌های `USER CODE BEGIN … END` فایل `main.c` بمانند (مثلاً بین `USER CODE BEGIN 4` و `END 4`). نرم‌افزار CubeMX هر بار که کد را دوباره تولید کند هرچه بیرون این بخش‌هاست پاک می‌کند. تابع `Start_lcd_Task` به‌صورت `__weak` تعریف شده، یعنی تعریف شما جایگزین نسخه‌ی پیش‌فرض می‌شود.

> **یادآوری مهم برای کامپایل**
> اگر هنگام کامپایل خطای «conflicting types for LCD_DelayUs» دیدید، یعنی پروتوتایپ بالای فایل (`uint16_t`) با تعریف جدید (`uint32_t`) نمی‌خواند. هر دو را `uint32_t` کنید.

۸. الگوی ساخت برنامه‌های دیگر

هر برنامه‌ی نمایشی را می‌توان با همین چهار گام ساخت. پروژه‌های بزرگ‌تر فقط تکرار همین الگوی ساده هستند:

۸-۱ فرمول چهارمرحله‌ای

1. **پیام‌ها را تعریف کنید:** چه رویدادهایی وجود دارد؟ به هرکدام یک عدد بدهید.
2. **فرستنده بنویسید:** هر جا که رویداد رخ می‌دهد (دکمه، سنسور، تایمر)، فقط پیام در صف بگذارید.

3. **گیرنده را در lcd_Task تکمیل کنید:** یک `case` برای هر پیام که `line1` و `line2` را تنظیم کند.
4. **Refresh را تغییر ندهید:** همان قبلی وظیفه‌ی نوشتن را انجام می‌دهد.

۸-۲ مثال الف: نمایش هر دو سطر

فقط کافی است `case` جدید بنویسید، مثلاً:

```
case 4: line1 = "Temperature:";
        line2 = "Sensor OK";  break;
```

۸-۳ مثال ب: نمایش یک عدد (شمارنده یا دما)

برای عدد نیاز به یک «متن ساخته‌شده در زمان اجرا» داریم. چون `line2` فقط آدرس نگه می‌دارد، باید متن را در یک آرایه‌ی سراسری بسازیم که همیشه سر جایش بماند:

```
static char numText[17];                // lives forever

static void NumberToText(uint32_t v, char *out)
{
    char tmp[11]; uint8_t n = 0;
    if (v == 0) tmp[n++] = '0';
    while (v > 0) { tmp[n++] = '0' + (v % 10); v /= 10; }
    for (uint8_t i = 0; i < n; i++) out[i] = tmp[n-1-i];
    out[n] = '\0';
}

/* inside the switch: */
case 5: NumberToText(counter, numText);
        line2 = numText;  break;
```

توضیح: عدد را با تقسیم‌های پی‌درپی بر ۱۰ به رقم‌ها می‌شکنیم (از آخر به اول) و سپس برعکس می‌کنیم. حرف `'0' + رقم` کد حرف رقم را می‌سازد. از `sprintf` عمداً استفاده نکردیم، چون پشته‌ی زیادی می‌گیرد و lcd_Task پشته‌ی کوچکی دارد.

۸-۴ مثال پ: اضافه کردن دکمه‌ی چهارم

1. در TouchGFX Designer یک دکمه بگذارید و یک «Interaction» با نام مثلاً `btnhello_func` «Call virtual function» بسازید. سپس Generate Code بزنید.
2. در `screenView.hpp` آن را `override` کنید و در `screenView.cpp` بنویسید: مقدار ۴ را به `prompt` بدهید.
3. در `Model::SetPrompt` یک `case 4` اضافه کنید که `osMessageQueuePut` را صدا بزند. (یا ساده‌تر: هر عدد را بفرستید.)
4. در `Start_lcd_Task` یک `case 4` بنویسید که `line1` را تعیین کند.

۸-۵ مثال ت: داده از سنسور (بدون لمس)

یک تسک جدید می‌سازید که هر ثانیه سنسور را می‌خواند و مقدار را مستقیماً در همان صف می‌اندازد. چون صف `uint32_t` می‌پذیرد، می‌توانید نوع و مقدار را در یک عدد بسته‌بندی کنید، مثلاً `(type << 24) | value`. گیرنده با `command >> 24` نوع و با `command & 0xFFFFFF` مقدار را جدا می‌کند. تسک LCD هیچ تغییری جز یک case ندارد.

> **سه قانون برای همه‌ی برنامه‌هایتان**
> ۱) فقط یک تسک با یک قطعه‌ی سخت‌افزار کار کند و دیگران پیام بفرستند. ۲) در تسک رابط کاربری هرگز کار طولانی یا تأخیر انجام ندهید. ۳) هر چیزی را که نمی‌توانید تشخیص بدهید خراب شده، به‌طور دوره‌ای بازسازی کنید.

۹. عیب‌یابی و واژه‌نامه

۹-۱ جدول عیب‌یابی سریع

علامت	علت محتمل	اقدام
صفحه کاملاً خالی یا فقط مربع‌های سیاه	تنظیم کنتراست (پتانسیومتر پشت LCD)، تغذیه، یا راه‌اندازی انجام نشده	کنتراست را بچرخانید؛ ولتاژ ۵ ولت را بسنجید

حروف آشغال یا ناخوانا	نیبل‌ها جابه‌جا شده‌اند یا زمان‌بندی اشتباه است	Refresh (که Init دارد) باید آن را درست کند؛ سیم‌ها و نویز را بررسی کنید
متن قدیمی باقی می‌ماند	پر کردن با فاصله اجرا نمی‌شود	مطمئن شوید از LCD_PrintLine استفاده می‌کنید
فقط ۸ حرف اول یک سطر دیده می‌شود	بعضی ماژول‌ها خانه‌ها را در آدرس‌های ناپیوسته نگه می‌دارند	مشخصات ماژول را بخوانید
دکمه کار می‌کند ولی LCD تغییر نمی‌کند	پیام نمی‌رسد یا case مربوطه نیست	در `SetPrompt` و `switch` نقطه‌ی توقف (Breakpoint) بگذارید
یک پلک‌زدن ریز هر ۲ ثانیه	طبیعی است؛ Init صفحه را پاک می‌کند	مهلت ۲۰۰۰ را بزرگ‌تر کنید
تسک اصلاً اجرا نمی‌شود	حافظه‌ی heap کم است (فقط ۱۵ کیلوبایت)	اندازه‌ی heap را در FreeRTOSConfig.h بزرگ‌تر کنید

۹-۲ ابزار پیشنهادی برای یادگیری

- برای دیدن زنده‌ی متغیرها از Live Expressions در STM32CubeIDE استفاده کنید (مثلاً line1 را مشاهده کنید).
- برای یافتن مشکل پشته از `uxTaskGetStackHighWaterMark` استفاده کنید. این تابع فعال شده و می‌گوید تسک چقدر از پشته‌اش را در بدترین حالت مصرف کرده است.
- اگر یک مشکل سخت‌افزاری مشکوک است، یک اسیلوسکوپ یا آنالایزر منطقی روی EN و SCK بگذارید و ببینید پالس‌ها شکل درست دارند یا نه.

۹-۳ واژه‌نامه

اصطلاح	معنی ساده
Microcontroller	کامپیوتر کوچک داخل یک تراشه (مثل STM32)
Task	یک برنامه‌ی مستقل با حلقه‌ی خودش داخل سیستم‌عامل
Queue	صندوق پیام بین تسک‌ها
Nibble	چهار بیت (نیمی از یک بایت)
SPI	پروتکل ساده‌ی ارسال بیت‌ها با کلاک و داده
74HC595	تراشه‌ای که داده‌ی سریال را به هشت خروجی موازی تبدیل می‌کند
HD44780	تراشه‌ی کنترل‌کننده‌ی استاندارد داخل LCD کاراکتری
RS / EN	پایه‌های «دستور یا داده» و «تأیید» روی LCD
Pointer	متغیری که آدرس یک مقدار را نگه می‌دارد
HAL	کتابخانه‌ی آماده‌ی ST برای کار با سخت‌افزار
TouchGFX	ابزار طراحی صفحه‌های گرافیکی و لمسی برای STM32
Self-healing	طراحی‌ای که خرابی را خودش جبران می‌کند

گام بعدی پیشنهادی

اول فقط یک متن جدید به switch اضافه کنید و ببینید کار می‌کند. بعد یک دکمه‌ی چهارم بسازید. بعد عدد را نمایش دهید. هر کدام ده دقیقه وقت می‌گیرد، ولی پس از آن الگوی «پیام → تسک → Refresh» در ذهن شما جا می‌افتد و می‌توانید هر برنامه‌ی مشابهی را بنویسید.